



# LexIA Maroc

Plateforme SaaS de veille juridique multi-RAG pour le droit marocain

## Documentation technique du projet

Architecture, étapes de réalisation et explication détaillée des algorithmes

|                     |                                                                                      |
|---------------------|--------------------------------------------------------------------------------------|
| Auteur              | N'Guémawen N'DJINILA                                                                 |
| Cadre               | Stage de fin d'année — Transformation Digitale Industrielle                          |
| Organisme           | Zenithsoft, Rabat                                                                    |
| Version applicative | LexIA Maroc v2.0                                                                     |
| Corpus indexé       | 6 353 passages juridiques (droit du travail, fiscal, sociétés, données personnelles) |

# Sommaire

1. Contexte et objectifs
2. Architecture globale et principes de conception
3. Pipeline d'ingestion des textes juridiques
4. Le cœur du système : le pipeline multi-RAG, algorithme par algorithme
5. Génération de la réponse (LLM)
6. Fonctionnalités métier
7. Sécurité et authentification
8. Modèle d'abonnement freemium
9. Déploiement en production et tests
10. Bilan, limites et perspectives

# 1. Contexte et objectifs

Le droit marocain est **dispersé** : Bulletin officiel, codes (travail, commerce, impôts), dahirs, lois-cadres, décrets et circulaires. Retrouver la disposition applicable à une situation précise, avec sa référence exacte, demande une expertise et un temps considérables. Les grands modèles de langage (LLM) généralistes ne résolvent pas ce problème : ils **hallucinent** des articles, ignorent les textes marocains récents et ne fournissent aucune source vérifiable.

**LexIA Maroc** répond à ce besoin par une architecture **multi-RAG** (Retrieval-Augmented Generation) : le modèle de langage ne répond jamais « de mémoire », mais uniquement à partir de passages de textes officiels **réellement récupérés** dans un corpus indexé, en citant systématiquement l'article, le document et la page. C'est la différence fondamentale entre un chatbot généraliste et un assistant juridique de confiance.

## Objectifs fonctionnels

- Répondre en langage naturel (français et arabe) sur quatre domaines : droit du travail, fiscal, des sociétés et protection des données personnelles.
- Ancrer chaque réponse sur des sources officielles et les citer précisément (article + page).
- Ne jamais affirmer ce qui n'est pas dans les textes récupérés (garde anti-hallucination).
- Adapter le registre de la réponse au profil de l'utilisateur (étudiant, particulier, juriste, avocat, entreprise).
- Compléter le chat par des outils métier : calculateurs, audit de contrat, génération de documents, comparaison de versions.

Le projet s'inscrit dans un stage de fin d'année (Zenithsoft, Rabat) et suit un cahier des charges de dix-huit exigences fonctionnelles (BF-01 à BF-18), en visant une qualité produit et non un simple prototype académique.

## 2. Architecture globale et principes de conception

Le système est organisé en **quatre couches applicatives** plus une couche transverse (sécurité, persistance), avec une **règle de dépendance stricte** : les couches métier dépendent d'un noyau neutre, jamais l'inverse. Cette discipline évite les couplages circulaires et permet d'ajouter un domaine juridique ou une fonctionnalité sans refonte.

*core ← ingestion / processing / retrieval / generation ← api*

*Sens des dépendances. « core » ne dépend de personne ; « api » orchestre tout.*

### Rôle de chaque couche

| Couche     | Responsabilité                                                                                                        |
|------------|-----------------------------------------------------------------------------------------------------------------------|
| core       | Fondations neutres : registre des domaines, configuration, modèles de base de données, sécurité, offres d'abonnement. |
| ingestion  | Acquisition des textes : téléchargement (scrapers), extraction PDF/OCR, pipeline d'indexation, veille planifiée.      |
| processing | Transformation : découpage en chunks, classification par domaine, calculateurs déterministes.                         |
| retrieval  | Recherche : embeddings, BM25, expansion, fusion RRF, reranking, routage, score de confiance.                          |
| generation | Production de la réponse : construction du prompt, client LLM avec cascade de secours.                                |
| api        | Exposition HTTP (FastAPI) : authentification, chat en streaming, recherche, outils métier, abonnement.                |

### Pile technique

- **Backend** : Python 3.11, FastAPI, SQLAlchemy, base Chroma (vectorielle, une collection par domaine), SQLite/PostgreSQL.
- **Modèles ML** : sentence-transformers (embeddings + cross-encoder), rank-bm25, Tesseract (OCR fr+ar).
- **Génération** : Groq (llama-3.3-70b-versatile), avec OpenAI / Gemini / OpenRouter en fournisseurs alternatifs.
- **Frontend** : React 18, servi en production par nginx.
- **Industrialisation** : Docker Compose (API, frontend, service de veille), JWT + bcrypt, tests pytest.

**Pourquoi multi-RAG plutôt qu'un LLM seul ?** Un LLM seul est une mémoire figée et faillible. Le RAG sépare la **connaissance** (le corpus officiel, à jour, vérifiable) de la **capacité de rédaction** (le LLM). Le modèle ne fait que reformuler et structurer des extraits qu'on lui fournit — d'où l'ancrage, la citation des sources et la maîtrise des hallucinations.

## 3. Pipeline d'ingestion des textes juridiques

Avant toute recherche, les textes bruts (PDF officiels) sont transformés en unités indexables. Le pipeline d'ingestion enchaîne cinq étapes, depuis le PDF jusqu'aux index de recherche.

### 3.1 Extraction du texte (avec OCR de secours)

L'extraction utilise d'abord **PyMuPDF** pour lire le texte natif du PDF (rapide, fidèle). Lorsque le PDF est un **scan** (image sans couche texte), on bascule sur l'**OCR Tesseract** configuré pour le français et l'arabe (fra+ara). Un garde-fou limite le nombre de pages soumises à l'OCR (protection contre les documents volumineux), mais ne s'applique qu'à ce chemin coûteux : un PDF à texte natif de 700 pages (le Code Général des Impôts) est extrait intégralement.

### 3.2 Suivi de la page source

L'extracteur conserve les **offsets de page** : la position de caractère où commence chaque page. Après découpage, chaque chunk est relié à sa page d'origine par recherche de sous-chaîne. C'est ce qui permet la citation « *Code du travail 65-99, p. 34* » — un différenciateur fort vis-à-vis des chatbots juridiques concurrents.

### 3.3 Découpage en chunks

Un texte juridique n'est pas un flux homogène : il est structuré en articles et alinéas. Le chunker exploite cette structure avec des motifs (regex) propres à la rédaction marocaine (« Article N », « Art. N », « N — », « Chapitre », « Titre »...) pour découper **par article**. À défaut de structure détectable, il retombe sur un découpage par taille :

*taille cible = 800 caractères | chevauchement = 150 caractères*

Le chevauchement (*overlap*) évite qu'une information à cheval sur deux chunks soit perdue : les 150 derniers caractères d'un chunk sont répétés au début du suivant.

### 3.4 Snapshots de versions

À chaque ingestion, une **copie du texte intégral** est conservée avec son empreinte (hash). Ces instantanés alimentent la fonction de **comparaison de versions** (par exemple CGI 2024 vs CGI 2025) : le diff porte sur le texte complet, pas sur les chunks (impropres au diff car découpés).

### 3.5 Indexation double

Chaque chunk est indexé **deux fois**, car les deux recherches sont complémentaires :

- **Index vectoriel (Chroma)** : le chunk est transformé en vecteur sémantique (voir §4.3) et stocké dans la collection de son domaine.
- **Index lexical (BM25)** : un index en mémoire pour la correspondance exacte des termes (numéros d'article, mots rares).

## 4. Le cœur du système : le pipeline multi-RAG

C'est le composant central. Une question suit une chaîne de traitements dont chaque maillon a un rôle précis. Vue d'ensemble :

*question → routage de domaine → expansion → recherche hybride (dense + BM25)*  
*→ fusion RRF → reranking cross-encoder → score de confiance → génération*

Chaque étape est détaillée ci-après avec sa justification et sa formule.

### 4.1 Routage par domaine (classification)

Inutile d'interroger les quatre corpus si la question porte clairement sur le droit du travail. Le routeur classe la question par **comptage de mots-clés** associés à chaque domaine. Point technique subtil : la correspondance se fait à **limite de mot** (`\b` en regex). Sans cela, un mot-clé court comme « IS » (impôt sur les sociétés) ou « IR » (impôt sur le revenu) serait trouvé par simple sous-chaîne à l'intérieur de mots français courants — « préavis », « secrétaire » — et provoquerait un mauvais routage.

Le score de confiance du routage est la part du domaine dominant :

*confiance = (score du meilleur domaine) / (somme des scores de tous les domaines)*

Si la confiance dépasse le seuil de **0,3**, on cible le domaine dominant (et ses domaines complémentaires) ; sinon, par prudence, on interroge **tous** les corpus prioritaires. Mieux vaut une recherche plus large qu'une réponse manquée.

### 4.2 Embeddings denses et similarité sémantique

La recherche « dense » repose sur des **embeddings** : chaque texte est projeté dans un espace vectoriel où la **proximité géométrique traduit la proximité de sens**. Deux formulations différentes d'une même idée (« licenciement abusif » et « rupture injustifiée du contrat ») aboutissent à des vecteurs proches, là où une recherche par mots-clés échouerait.

Le modèle utilisé est **intfloat/multilingual-e5-large** (1024 dimensions, multilingue français/arabe). Il impose des **préfixes** distincts pour les requêtes et les passages, ce qui améliore nettement la qualité :

```
QUERY_PREFIX = "query: "      # préfixe des questions
DOC_PREFIX    = "passage: "    # préfixe des passages indexés
embedding = model.encode(prefix + texte, normalize_embeddings=True)
```

La proximité entre une requête **q** et un document **d** se mesure par la **similarité cosinus** :

$$\cos(q, d) = (q \cdot d) / (||q|| \times ||d||)$$

Comme les vecteurs sont **normalisés** à la génération (`normalize_embeddings=True`, donc  $||q|| = ||d|| = 1$ ), la similarité cosinus se réduit au simple **produit scalaire**, ce qui accélère la recherche. Chroma renvoie une distance ; on la convertit en similarité par `score = 1 - distance`.

### 4.3 Recherche lexicale BM25

La recherche dense a une faiblesse : sur un corpus juridique homogène, elle peut « lisser » les termes précis. Or « **article 62** » ou une référence exacte doivent être trouvés **à la lettre**. C'est le rôle de **BM25** (Best Matching 25), un modèle probabiliste de pertinence lexicale. Le score d'un document  $D$  pour une requête  $Q$  est :

$$\text{score}(D, Q) = \sum_i \text{IDF}(q_i) \cdot [f(q_i, D) \cdot (k_1 + 1)] / [f(q_i, D) + k_1 \cdot (1 - b + b \cdot |D| / \text{avgdl})]$$

$f(q, D)$  = fréquence du terme  $q$  dans  $D$  ;  $|D|$  = longueur de  $D$  ;  $\text{avgdl}$  = longueur moyenne des documents.

Le poids **IDF** (fréquence inverse de document) donne plus d'importance aux termes rares :

$$\text{IDF}(q_i) = \ln( (N - n(q_i) + 0,5) / (n(q_i) + 0,5) + 1 )$$

$N$  = nombre total de documents ;  $n(q)$  = nombre de documents contenant  $q$ .

Deux hyperparamètres règlent le comportement :  $k_1$  module la saturation de la fréquence (au-delà d'un certain nombre d'occurrences, un terme n'apporte plus grand-chose), et  $b$  contrôle la normalisation par la longueur (pénalise les documents longs). BM25 apporte donc la **précision lexicale** que le dense n'offre pas.

#### 4.4 Expansion de requête

Une même intention peut s'exprimer de plusieurs façons. Avant de chercher, le système génère quelques **variantes** de la question (reformulations, synonymes juridiques). Chaque variante produit sa propre liste de résultats, ce qui augmente le rappel (on rate moins de passages pertinents formulés autrement que la question d'origine).

#### 4.5 Fusion des résultats : Reciprocal Rank Fusion (RRF)

On dispose maintenant de plusieurs listes classées (dense et BM25, pour chaque variante). Comment les combiner alors que leurs scores ne sont **pas à la même échelle** (une similarité cosinus dans  $[0,1]$  et un score BM25 non borné) ? La réponse est **RRF**, qui ignore les scores bruts et ne combine que les **rangs** :

$$\text{RRF}(d) = \sum_{\text{listes}} 1 / (k + \text{rang}(d)) , \text{ avec } k = 60$$

$\text{rang}(d)$  = position du document  $d$  dans une liste (0 = premier). Constante  $k = 60$  (valeur usuelle).

Un document bien classé dans **plusieurs** listes cumule les contributions et remonte. L'astuce du **+k** au dénominateur atténue l'écart entre les toutes premières positions : passer du rang 0 au rang 1 change peu le score, ce qui rend la fusion robuste au bruit. RRF est simple, sans paramètre à entraîner, et remarquablement efficace pour fusionner des signaux hétérogènes — d'où son choix ici.

#### 4.6 Reclassement par cross-encoder (reranking)

Les embeddings de §4.2 sont produits par un **bi-encodeur** : la requête et le passage sont encodés **séparément**, puis comparés. C'est rapide (les passages sont pré-encodés) mais approximatif, car l'encodage d'un passage ignore la requête. Pour affiner, on applique un **cross-encodeur** (**ms-marco-MiniLM**) aux meilleurs candidats seulement :

- **Bi-encodeur** :  $\text{encode}(q)$  et  $\text{encode}(d)$  indépendamment  $\rightarrow$  cosinus. Rapide, sert au premier tri sur tout le corpus.

- **Cross-encodeur** : traite la paire (q, d) **ensemble** dans un même passage avant du modèle → score de pertinence bien plus fin. Coûteux, réservé au re-tri des ~10 meilleurs.

Le cross-encodeur renvoie un **logit** brut (non borné). On le ramène dans [0,1] par une fonction **sigmoïde**, pour obtenir un score de pertinence affichable :

$$\sigma(s) = 1 / (1 + e^{-s})$$

Un **bonus de correspondance d'article** complète le cross-encodeur : si la question cite « article 52 » et que le passage provient justement de l'article 52, son score est multiplié par 1,5 (plafonné à 1). Le cross-encodeur seul ne privilégie pas toujours la correspondance exacte de référence — ce bonus la garantit.

## 4.7 Score de confiance

La qualité d'une réponse dépend de la qualité des passages trouvés. On expose donc un **score de confiance**, égal au score de reranking du **meilleur** passage retenu (déjà normalisé en [0,1]). Il est traduit en label qualitatif — l'utilisateur voit « Confiance élevée », jamais un pourcentage trompeur :

| Score de confiance  | Label affiché                                   |
|---------------------|-------------------------------------------------|
| score ≥ 0,80        | élevé                                           |
| 0,60 ≤ score < 0,80 | moyen                                           |
| 0,40 ≤ score < 0,60 | faible                                          |
| score < 0,40        | insuffisant → message « aucun texte pertinent » |

## 4.8 Filtre de pertinence de la recherche par référence

La recherche par référence (trouver un texte par son numéro) a nécessité un garde-fou spécifique. Lorsqu'aucun mot-clé ne correspond, le repli vectoriel renvoie **toujours** les voisins les plus proches — même hors sujet, car sur un corpus juridique tout se ressemble (similarité ~0,8). Une requête comme « note circulaire TVA » (aucune circulaire n'est indexée) remontait ainsi cinq passages sans rapport. La correction impose un **recouvrement lexical** : un résultat vectoriel n'est retenu que s'il contient réellement au moins un mot significatif de la requête ; sinon la liste est vide et l'interface l'annonce honnêtement.



## 5. Génération de la réponse (LLM)

Les passages retenus sont assemblés dans un **prompt** soumis au modèle de langage. Le modèle ne fait que **rédigier** à partir de ce contexte — il ne puise pas dans sa mémoire.

### 5.1 Fournisseur et cascade de secours

Le fournisseur par défaut est **Groq** (llama-3.3-70b-versatile) : gratuit et très rapide (~7-8 s). Une **cascade de secours** bascule automatiquement sur le modèle suivant en cas d'erreur **récupérable** — la distinction est essentielle :

| Type d'erreur          | Comportement de la cascade                                                   |
|------------------------|------------------------------------------------------------------------------|
| 401 (clé invalide)     | Fatal — changer de modèle n'y changera rien, on arrête.                      |
| 402, 408, 429, 500-504 | Récupérable — crédits/limite/timeout/panne : on tente le modèle suivant.     |
| Coupure réseau / DNS   | Récupérable — l'incident peut être transitoire ou localisé à un fournisseur. |

En mode **streaming** (affichage progressif des tokens via SSE), la bascule n'est possible qu'**avant** le premier token : une fois le flux commencé, rebasculer casserait l'affichage.

### 5.2 Adaptation au profil (prompt engineering)

Le prompt système est composé dynamiquement selon le rôle de l'utilisateur — cinq **personas**, chacun avec un ton et une structure de réponse propres :

| Profil           | Registre et structure de la réponse                                                    |
|------------------|----------------------------------------------------------------------------------------|
| Étudiant         | Pédagogique, définitions des termes. Sections « Analyse » / « Ce qu'il faut retenir ». |
| Particulier      | Vulgarisé, accessible, orienté action concrète.                                        |
| Juriste / Avocat | Technique, références d'articles exactes, argumentaire.                                |
| Entreprise       | Orienté dirigeant. Sections « Obligations » / « Risques » / « Actions ».               |

### 5.3 Garde anti-hallucination

Des règles strictes encadrent la génération : le service traite **exclusivement** le droit marocain (le modèle ne doit jamais présenter un texte comme étranger — incident réel : la loi 31-13 avait été qualifiée « de loi française ») ; aucune affirmation ne doit sortir des sources fournies ; et les réponses « cul-de-sac » sont bannies — à défaut de trouver, le système indique ce que les corpus couvrent et invite à reformuler, plutôt qu'un « aucune information » sec. Enfin, un **budget de contexte** (comptage de tokens via tiktoken) élague les passages excédentaires pour ne jamais dépasser la fenêtre du modèle.

## 6. Fonctionnalités métier

Autour du chat, des outils à forte valeur transforment l'assistant en véritable plateforme.

### 6.1 Calculateurs juridiques (déterministes)

Contrairement au chat, ces calculs **n'utilisent pas de LLM** : ils sont déterministes et reproductibles, fondés sur le barème légal, avec référence à l'article. Trois calculateurs : **indemnité de licenciement** (barème progressif de l'article 53 : 96 h de salaire par an les 5 premières années, puis 144 h, 192 h, 240 h), **préavis légal** (article 51) et **salaire net** (CNSS 4,48 % plafonnée, AMO 2,26 %, IR au barème progressif).

### 6.2 Audit de contrat

L'utilisateur importe un contrat ; le système extrait son texte, **récupère les articles de loi pertinents** puis génère un rapport structuré (points de conformité, risques, clauses manquantes). Point clé : l'audit interroge le corpus **matière par matière** (période d'essai, préavis, salaire, congés, rupture, durée du travail) plutôt qu'en une requête unique — sans quoi des articles décisifs (comme l'article 14 sur la durée maximale de la période d'essai) n'étaient jamais récupérés, et l'audit concluait à tort à la conformité.

### 6.3 Génération de documents juridiques

Quatre modèles déterministes conformes au Code du travail — **attestation de travail** (art. 72), **mise en demeure** pour salaire impayé, **contrat CDI** (art. 14, 51, 53...) et **reçu pour solde de tout compte** (art. 73-74) — remplis par formulaire et exportables en **Word (DOCX)** et **PDF**. La même trame neutre est rendue dans les deux formats.

### 6.4 Autres outils

- **Comparaison de versions** : diff ligne à ligne entre deux instantanés d'un même texte.
- **Recherche par référence** : retrouver un texte par son numéro exact (loi 09-08, article 62, dahir 1-72-184), avec le filtre de pertinence du §4.8.
- **Veille réglementaire** : un service planifié détecte les nouveaux textes et notifie ; un bandeau « derniers textes intégrés » anime la page d'accueil.
- **Support de l'arabe** : requêtes et réponses en arabe, interface RTL.

## 7. Sécurité et authentification

- **JWT + bcrypt** : mots de passe hachés (bcrypt, 12 rounds) ; jeton d'accès de courte durée gardé **en mémoire** côté client (jamais dans localStorage, pour résister au vol par XSS).
- **Refresh token en cookie httpOnly** : invisible au JavaScript, transmis automatiquement ; en production, drapeau **Secure** (HTTPS obligatoire).
- **Rotation du refresh token** : chaque rafraîchissement révoque l'ancien jeton et en émet un nouveau. Un correctif « single-flight » mutualise les rafraîchissements concurrents (React StrictMode en lançait deux, dont le second échouait et déconnectait l'utilisateur).
- **Rate limiting** : 60 requêtes / 60 s par IP, en défense contre l'abus.
- **RBAC** et verrouillage de fonctionnalités par offre d'abonnement (voir §8).
- **En-têtes de sécurité** (CSP, HSTS, X-Frame-Options...) appliqués par nginx.

## 8. Modèle d'abonnement freemium

La plateforme intègre une mécanique d'abonnement complète — offres, quotas, compteur d'usage, paywall — prête à être reliée à un prestataire de paiement (**Stripe** international ou **CMI** au Maroc) sans autre modification : l'activation d'offre passe par un point d'entrée unique aujourd'hui « simulé ».

| Offre   | Prix   | Contenu                                                                                           |
|---------|--------|---------------------------------------------------------------------------------------------------|
| Gratuit | 0 MAD  | 20 questions / mois. Chat sourcé, calculateurs, recherche par référence, comparaison de versions. |
| Pro     | 99 MAD | Questions illimitées + audit de contrat + génération de documents + export (JSON/Word).           |

Chaque fonctionnalité à forte valeur déclare la clé qui la déverrouille ; l'API refuse (code HTTP 402) l'accès si l'offre de l'utilisateur ne l'inclut pas ou si son quota mensuel est épuisé. Le compteur d'usage est vérifié à chaque question.

## 9. Déploiement en production et tests

### 9.1 Conteneurisation

Trois services orchestrés par Docker Compose : l'**API** (uvicorn, 2 workers, utilisateur non-root, modèles ML préchargés dans l'image pour éviter un téléchargement de 2,2 Go au premier démarrage), le **frontend** (build React statique servi par nginx) et le **watcher** (veille planifiée). Des *healthchecks* gouvernent l'ordre de démarrage.

### 9.2 Configuration de production

- ENVIRONMENT=production désactive la documentation d'API (/docs) et active les cookies Secure.
- L'API refuse de démarrer si SECRET\_KEY n'a pas été changée.
- HTTPS terminé par un reverse proxy ; CORS restreint à l'origine du frontend.
- Secrets (clés LLM, SMTP) uniquement dans un fichier .env hors dépôt Git.

### 9.3 Qualité et tests

Le projet est couvert par **105 tests** (unitaires et d'intégration) : classifieur, chunker, fusion RRF, calcul de confiance, calculateurs, filtres de recherche, génération de documents, flux d'authentification, etc. Plusieurs bugs réels n'ont été détectés qu'en **exécutant les vrais parcours** (et non par les seuls tests) : recherche d'historique cassée, session perdue au rechargement, recherche par référence hors sujet, audit passant à côté d'une non-conformité — tous corrigés et couverts par des tests de non-régression.

## 10. Bilan, limites et perspectives

LexIA Maroc dépasse le stade du prototype : architecture propre en couches, pipeline multi-RAG complet (dense + BM25 + RRF + reranking + confiance), réponses sourcées à la page, outils métier, sécurité soignée, abonnement freemium et déploiement conteneurisé documenté.

### Limites connues

- **Corpus jurisprudence vide** : les décisions de la Cour de cassation ne sont pas disponibles sous forme d'un recueil PDF exploitable proprement.
- **OCR des scans** : l'installation de Tesseract sur le poste de développement a été bloquée par les droits administrateur ; les dahirs/arrêtés scannés ne sont donc pas encore océrisés.
- **Paiement non branché** : la mécanique d'abonnement est complète mais le prestataire réel reste à connecter.

### Perspectives

- Brancher le paiement réel (Stripe ou CMI) sur le point d'activation déjà prévu.
- Ajouter les conditions générales et la politique de confidentialité (conformité à la loi 09-08).
- Sourcer et indexer un corpus de jurisprudence.
- Observabilité (latence, coût par requête) et détection automatique de la langue.

---

Document technique généré pour le projet LexIA Maroc — Stage de fin d'année, Zenithsoft (Rabat). Les réponses fournies par la plateforme sont informatives et ne constituent pas un avis juridique professionnel.